Home Assignment – Full-Stack Developer

Goal

Build a small **job-based** web crawler system that demonstrates:

- **Hands-on backend + frontend delivery**
- **Event-driven microservices** (RabbitMQ or Kafka)
- **.NET 8 / ASP.NET Core** REST APIs
- **React** UI patterns
- **SQL persistence + performance considerations**
- **Engineering practices** (tests, observability, clean architecture, CI/CD mindset)

What to Build

A system that allows a user to submit a **crawl job** for a given website URL, processes it asynchronously, stores results, and presents them in a React UI.

High-level user flow

1. User enters a URL and clicks **Start Crawl**.
2. UI immediately shows a **Job Details** page with a **jobId** and status.
3. While running, UI shows **progress** and updates until completion.
4. When complete, UI displays a **tree view** of discovered pages and the computed **Domain Link Ratio** (explained below) for each page.
5. UI provides a **History** screen listing previous crawl jobs with their status and timestamps.

Architecture Requirements

Implement at least **two processes** (separate projects/containers):

Service A — Crawl API (Orchestrator)

Responsible for:

- Creating jobs
- Serving job status, results and history to the UI

Service B — Crawl Worker

Responsible for:

- Fetching pages, parsing links, calculating metrics
- Updating job progress/state

Message Broker

Use **RabbitMQ or Kafka**.

- Provide local setup via **Docker Compose**.

Database

Use any SQL DB (e.g., PostgreSQL, SQL Server, MySQL).

- The system must persist jobs, pages, and edges (parent → child). You decide where persistence is implemented.

Functional Requirements

1) Job lifecycle

A crawl job has:

- **jobId** (GUID)
- input URL
- status: `Pending | Running | Completed | Failed | Canceled`
- timestamps: created, started, completed
- optional failure reason

API must support (you define the URL structure):

- Create a new job
  - request: `{ url: string, maxDepth?: number (default 2) }`
  - response: `{ jobId }`

- Read job status and summary
- Read hierarchical tree result
- List job history (paginated, most recent first)

Optional:

- Cancel a job

2) Crawling rules

- Crawl **HTML pages only**.
- Default crawl depth: **2**.
- For each page, collect:
  - absolute URL
  - list of discovered outgoing links
  - **Domain Link Ratio** (see below)
- Handle:
  - relative links
  - anchor links (ignore fragment when normalizing)
  - duplicate URLs (do not reprocess the same normalized URL within a job)

Constraints:

- Include a **max pages per job** safety limit (e.g., 200) with a sensible default.
- Use timeouts and retries for HTTP (reasonable defaults).

3) Domain Link Ratio

For each crawled page, compute:

```
Domain Link Ratio = (# outgoing links that remain within the starting domain) / (total # outgoing links)
```

Notes:

- Define “starting domain” as the host of the initial job URL.
- Ignore `mailto:`, `tel:`, and non-http(s) schemes.
- If a page has zero outgoing links, ratio is `0`.

Event-Driven Requirements (Core)

Demonstrate practical message-driven design:

Required behaviors

- **At-least-once processing** assumption: your worker must be **idempotent**.
  - Same task delivered twice must not create duplicate page/edge rows.
- **Retries** for transient failures.
- **Dead-letter queue/topic** for poison messages.

What to document in README

- Message schema
- Idempotency strategy (unique keys, upserts, de-dup tables, etc.)
- Retry policy and what qualifies as transient
- DLQ handling (what gets routed there)

Frontend Requirements (React)

Build a small React app (Vite/CRA/Next—any is fine) with:

Screens

1. **Start Crawl**
   a. input URL
   b. optional: maxDepth
   c. submit starts a job and navigates to Job Details
2. **Job Details**
   a. job status and timestamps
   b. progress indicator (polling is acceptable; SSE/SignalR is a stretch)
   c. once completed: show **tree view**
3. **History**
   a. list of previous jobs
   b. click to open Job Details

UI expectations

- Clear loading and error states

- No need for fancy styling; clarity > design

Engineering Quality Requirements

Tests

Minimum expectations:

- Unit tests for:
  - URL normalization (relative links, fragments)
  - Domain Link Ratio calculation
- One integration-style test using local HTML fixtures (recommended)

Observability

- Structured logs (include jobId/correlationId)
- Health endpoint(s) for API and worker

Clean Architecture / Maintainability

- Reasonable separation of concerns (API, domain, infrastructure)
- Meaningful naming, small methods, clear error handling

Delivery

- Provide `docker-compose.yml` that runs:
  - Crawl API
  - Worker
  - DB
  - RabbitMQ or Kafka
- Provide a README with:
  - how to run locally
  - key architectural choices
  - known limitations

Optional:

- CI pipeline (GitHub Actions): build + test

Evaluation Rubric

We will evaluate along these dimensions:

1. **Correctness & edge cases** (30%)
2. **Architecture & extensibility** (25%)
3. **Event-driven robustness** (idempotency/retries/DLQ) (15%)
4. **SQL model & performance thinking** (15%)
5. **Code quality, tests, observability, documentation** (15%)

Guidance

We recommend to use a **vibe coding** approach for this assignment, but ensure you can explain your implementation. You should allocate **up to 4 hours** of dedicated effort to this task, even though a comprehensive implementation would take significantly longer. The intention is to observe your priorities and decision-making under time constraints. After completing the assignment, please provide a summary explaining what you implemented, the choices you made, and any trade-offs or areas you prioritized during development.

In your README, briefly describe:

- what you chose to implement first (and why)
- what you cut (and why)
- what you would do next if you had more time

Submission

Provide:

- a repo or zip
- instructions to run
- any assumptions and trade-offs in the README